

# 算法工程师 CS 软件工程技能包 2026

面向大模型、搜索、广告、推荐与算法平台工程师。把真实生产中的代码、数据、训练、评测、服务、实验、稳定性、安全和成本能力整理成一套可执行的学习与交付标准。

作者: Winston  
资料口径: 2026-05-31  
形态: 长文档 / 技能地图 / 生产流程 / 检查表

判断一名算法工程师是否成熟，不看他会多少模型名词，而看他能不能把一次改动带过真实生产链路。

顶尖团队的共同点不是崇拜复杂算法，而是把复杂算法放进可复现、可回滚、可监控、可解释、可审计、成本可控的工程系统。

这份技能包按生产责任拆解：从 Git 变更、Hive 离线表、Spark 作业、Kafka 流、训练评测、模型服务、A/B 实验到事故复盘。

12

技能域

8

标准流程

7

岗位包

90

天训练路线

# 00 目录与阅读方法

先看链路，再看工具  
最后用检查表训练

## 01 核心结论

算法工程师的真实交付面是什么

## 03 技能地图

12 个技能域与掌握标准

## 05 公司标杆

Google、Meta、OpenAI、Anthropic、NVIDIA、ByteDance

## 07 标准流程

PR、离线特征、训练、上线、实验、故障处理

## 09 检查表

合并前、训练前、上线前、事故中

## 11 命令速查

Git、Hive、Spark、K8s、GPU、日志排查

## 02 生产链路

从需求到事故复盘的端到端路径

## 04 工具包

Git、Hive、Spark、Kafka、K8s、GPU、Serving

## 06 岗位技能包

LLM、RAG、搜索、广告、推荐、平台

## 08 成长路线

30 / 60 / 90 天与 6 个月进阶

## 10 反模式

真实团队里最常见的工程债

## 12 交付模板

PR、特征、上线、事故复盘的最小文档

**阅读顺序：**如果你是学生或转岗者，先读 01、02、03、08。如果你已经在做算法业务，直接从 07 和 09 开始，把自己的项目逐项对照。如果你要招人或带团队，重点看 05、06、09、10。

### 不是工具百科

工具名会变，但生产问题稳定存在：数据会脏、标签会漂、特征会穿越、模型会退化、服务会超时、实验会误判、GPU 会浪费。

### 不是纯算法路线

算法工程师必须能解释模型指标，也要能解释代码 diff、数据血缘、任务调度、资源消耗、线上延迟和回滚路径。

### 不是单一大厂模板

正文抽取公开资料里的共性标准，不声称复刻任何公司的非公开制度。落地时必须服从本团队合规与平台规则。

# 01 | 核心结论

算法价值通过工程系统兑现

## 一个成熟算法工程师的工作定义

算法工程师不是只训练模型的人，而是把数据、模型和产品决策接入可靠系统的人。他需要能提出模型假设，也要能让这个假设在离线数据、线上流量、服务延迟、实验统计和事故处理里站得住。

Google 的 ML 工程实践强调先建稳健基础设施和简单模型；Meta 的推荐系统鲁棒性材料强调模型、特征、训练数据、校准和可解释性的全链路防护；Anthropic 与 OpenAI 的公开岗位材料把大规模分布式系统、评测、监控、回滚、可靠性作为 ML infra 的核心能力；NVIDIA 的文档把 GPU 编程、性能分析、推理服务和模型优化变成硬门槛；ByteDance 的推荐开源系统展示了实时训练、在线服务和大规模推荐系统的工程面。

一句话标准：能独立把一个算法改动从问题定义推进到可复现离线实验、可审查代码、可回滚上线、可观测运行、可解释复盘，才算具备生产级算法工程能力。

## 四个层级的能力

| 层级   | 能力表现                           | 常见短板              |
|------|--------------------------------|-------------------|
| 会用   | 能跑 notebook、写 SQL、调模型 API、提交代码 | 离线结果不可复现，线上风险不可解释 |
| 能交付  | 能写 PR、接调度、做离线评测、发灰度实验          | 对异常数据、资源瓶颈和回滚不敏感  |
| 能负责  | 能设计指标、守住 SLO、处理事故、推动治理         | 跨团队协作和系统边界判断不足    |
| 能建平台 | 能抽象工具、统一标准、降低团队试错成本            | 容易过度平台化，忽视业务反馈    |

## 能力结构不是线性的

|                                             |                                                   |                                       |                                                   |                                        |
|---------------------------------------------|---------------------------------------------------|---------------------------------------|---------------------------------------------------|----------------------------------------|
| <b>代码</b><br>Git、PR、review、测试、CI、可读性、依赖与发布。 | <b>数据</b><br>Hive、Spark、Flink、Kafka、表治理、质量、血缘、隐私。 | <b>模型</b><br>训练、评测、复现、分布式、调参、模型注册和版本。 | <b>服务</b><br>Docker、K8s、RPC、Serving、缓存、限流、延迟、扩缩容。 | <b>治理</b><br>实验、监控、回滚、事故、安全、成本、文档和责任人。 |
|---------------------------------------------|---------------------------------------------------|---------------------------------------|---------------------------------------------------|----------------------------------------|

## 真实生产里的高频问题

- 离线 AUC 涨了，但线上 CTR、GMV、留存或负反馈没有改善。
- 训练样本表被上游改了，任务仍然成功，模型质量却慢慢下降。
- 特征在训练和服务使用了不同口径，出现 training-serving skew。
- 模型 snapshot 质量未通过就被服务加载，线上预测稳定性受损。
- LLM 推理成本没有预算边界，KV cache、batch、并发和限流设计缺失。
- 实验没有 guardrail 指标，短期点击提升换来投诉、延迟或生态损伤。
- 上线没有灰度、canary、回滚和 owner，问题只能靠人工排查。

学习原则：每学一个工具，都要问四个问题：它在链路里解决什么风险，输入输出是什么，失败时怎么发现，出问题时怎么回滚。

## 02 | 生产链路

算法改动的真实旅程

|                                                     |                                                     |                                                    |                                                     |                                                        |
|-----------------------------------------------------|-----------------------------------------------------|----------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------|
| <b>1. 需求与指标</b><br>定义用户问题、业务目标、主指标、护栏指标、反事实风险和回滚阈值。 | <b>2. 数据与标签</b><br>确定日志、样本、标签、时间窗、负采样、去重、延迟到达和数据权限。 | <b>3. 特征与召回</b><br>离线特征、实时特征、向量检索、倒排索引、缓存和训练服务一致性。 | <b>4. 训练与评测</b><br>固定代码、数据、配置、随机种子、模型版本，跑离线指标和诊断评测。 | <b>5. 服务与实验</b><br>模型注册、灰度、A/B、canary、SLO、日志、监控、告警和回滚。 |
|-----------------------------------------------------|-----------------------------------------------------|----------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------|

端到端责任表

| 阶段   | 主要产物                          | 算法工程师必须会问的问题                        | 常用工具                                   |
|------|-------------------------------|-------------------------------------|----------------------------------------|
| 问题定义 | 实验设计文档、指标树、风险清单               | 主指标是否可归因？护栏指标是否覆盖用户体验、收入、延迟和安全？     | docs、metrics platform、dashboard        |
| 样本构建 | Hive 表、Spark 任务、数据质量报告        | 样本是否漏日志？标签是否穿越？分区是否完整？负采样是否改变业务分布？  | Hive、Spark、Airflow、Argo                |
| 特征工程 | feature spec、离线和在线特征实现        | 训练和服务的特征口径是否一致？缺失、默认值、截断和归一化是否一致？   | Feast、Flink、Kafka、Redis、KV store       |
| 模型训练 | 训练配置、checkpoint、artifact、评测报告 | 能否复现？资源消耗是否合理？分布式训练失败后能否恢复？         | PyTorch、JAX、DeepSpeed、Ray、MLflow       |
| 离线评测 | 指标表、分桶诊断、错误案例集                | 是否只看平均值？是否覆盖冷启动、长尾、地域、设备、时间漂移和安全样本？ | notebook、SQL、eval harness、OpenAI Evals |
| 模型服务 | 镜像、服务配置、SLO、压测报告              | 延迟、吞吐、显存、批处理、降级、超时、缓存和并发边界是什么？      | Docker、K8s、Triton、vLLM、KServe          |
| 线上实验 | 实验配置、白名单、灰度曲线、决策记录            | 随机化单元是否正确？样本量是否足够？是否有污染和多实验干扰？      | A/B platform、feature flag、metrics      |
| 运行治理 | dashboard、alert、runbook、复盘文档  | 谁值守？什么阈值触发回滚？根因如何定位到数据、模型、服务或依赖？    | Prometheus、Grafana、OpenTelemetry、logs  |

最危险的误解：把“任务成功退出”当成“模型可上线”。在 ML 系统里，数据新鲜度、特征覆盖率、标签稳定性、校准、分布漂移和延迟退化都可能在任务成功时发生。

# 03 | 12 个技能域

从基础工程到模型治理

### 01 Linux 与命令行

sshtmuxrsyncjqcurl

能在远程机器排查进程、文件、端口、磁盘、网络、GPU、日志和权限问题。

### 02 Git 与协作

branchrebasebisectPR

能保持小变更、写清楚 PR、响应 review、定位回归、回滚风险改动。

### 03 编程语言

PythonC++JavaScalaGo

Python 做实验和平台胶水，C++/CUDA 做高性能，Java/Scala 常见于数据和服务栈，Go 常见于云原生服务。

### 04 测试与质量

pytestlinttypingCI

单测覆盖数据转换、特征函数、服务接口和固定模型输出，CI 防止低级回归。

### 05 SQL 与数仓

HivepartitionCBOlineage

能写可维护 SQL，理解分区、倾斜、小文件、窗口函数、UDF、血缘和权限。

### 06 批与流

SparkFlinkKafkaAirflow

能区分批处理、准实时、事件时间、watermark、checkpoint、backfill 和重放。

### 07 特征与标签

feature storelabelskew

能定义 owner、口径、默认值、监控、在线离线一致性和数据质量准入。

### 08 训练系统

PyTorchJAXDDPcheckpoint

能处理分布式训练、混合精度、恢复、指标记录、资源配置和实验追踪。

### 09 评测系统

offline evalslicecalibration

能构建稳定评测集，分桶诊断，处理 LLM judge 风险，建立上线阈值。

### 10 检索与排序

ANNFaissLuceneranker

能理解召回、粗排、精排、重排、多目标、探索利用和延迟预算。

### 11 Serving 与云原生

DockerK8sgRPCTritonvLLM

能做模型部署、批处理、动态扩缩容、压测、灰度、监控和降级。

### 12 安全与成本

PIISLSAOWASP LLMGPU cost

能管理数据权限、密钥、依赖、模型供应链、提示注入、滥用风险和单位经济账。

**掌握标准：** 不要用“我学过”来判断技能。用“我能否在生产约束下独立完成、出错后定位、影响用户前回滚、复盘后防再发”来判断。

# 04 | 高频工具包

真实工作里每天会碰到的东西

## 基础工程与调试

| 工具         | 必须掌握                                             | 生产判断                                  |
|------------|--------------------------------------------------|---------------------------------------|
| Git        | status、diff、log、rebase、cherry-pick、bisect、reflog | 能把一次模型改动拆成可审查的小 PR，能定位引入回归的 commit。   |
| Linux      | ssh、tmux、ps、lsof、du、df、top、journalctl            | 能在训练机和服务 Pod 上快速判断 CPU、内存、磁盘、端口和进程异常。 |
| Shell 数据工具 | rg、awk、sed、sort、uniq、jq、xargs                    | 能抽样检查日志和数据文件，不把生产排查完全依赖 notebook。     |
| Python 工程  | pytest、ruff、mypy、uv、pip、conda                    | 能把实验代码整理成可测试模块，依赖可锁定，环境可复现。           |
| 服务协议       | HTTP、gRPC、Thrift、Protobuf、JSON、OpenAPI           | 能解释超时、重试、幂等、限流、序列化成本和兼容性。             |

## 离线数据与实时流

| 工具    | 必须掌握                                                      | 生产判断                                  |
|-------|-----------------------------------------------------------|---------------------------------------|
| Hive  | 分区、动态分区、ORC/Parquet、Metastore、CBO、权限                      | 能识别分区缺失、倾斜 join、小文件、UDF 隐患和血缘风险。      |
| Spark | DataFrame、Spark SQL、shuffle、broadcast、cache、checkpoint、UI | 能通过 DAG 和 stage 判断慢任务、内存溢出、数据倾斜和资源浪费。 |
| Flink | event time、watermark、state、checkpoint、exactly-once        | 能解释乱序、迟到数据、状态膨胀、回放和在线特征延迟。            |
| Kafka | topic、partition、consumer group、offset、retention、schema    | 能处理积压、重平衡、重复消费、消息丢失和版本兼容。             |
| 调度    | Airflow、Argo、DAG、backfill、retry、SLA                       | 能设计补数策略和依赖告警，不让失败任务静默影响模型。            |

## 模型平台与服务

| 工具                | 必须掌握                                            | 生产判断                                                    |
|-------------------|-------------------------------------------------|---------------------------------------------------------|
| MLflow / registry | run、artifact、model registry、signature、promotion | 能追踪模型从实验到上线的代码、数据、配置和评测证据。                              |
| Feature Store     | entity、feature view、online store、offline store  | 能维护训练和服务一致性，区分 backfill、freshness 和 point-in-time join。 |
| Docker            | image、layer、entrypoint、multi-stage、registry     | 能构建可复现镜像，避免隐式依赖和过大镜像拖慢部署。                               |
| Kubernetes        | Pod、Deployment、Service、Secret、HPA、rollout、logs  | 能看懂服务状态、资源限制、探针、滚动发布和崩溃循环。                              |
| Serving           | Triton、TensorRT-LLM、vLLM、KServe、Ray Serve       | 能选择批处理、流式输出、KV cache、LoRA、多模型和 GPU 切分方案。                |

## GPU 与性能

| 工具            | 必须掌握                                             | 生产判断                           |
|---------------|--------------------------------------------------|--------------------------------|
| nvidia-smi    | 显存、利用率、进程、功耗、MIG、ECC                             | 能判断 GPU 闲置、显存碎片、进程泄漏和多租户干扰。    |
| Nsight        | timeline、kernel、CPU/GPU overlap、communication    | 能定位瓶颈在数据加载、通信、kernel、同步还是服务排队。 |
| CUDA / Triton | kernel、memory coalescing、occupancy、shared memory | 能读懂高性能算子优化报告，不把性能问题都归因于模型大。    |
| 压测            | p50/p95/p99、QPS、吞吐、排队、冷启动                        | 能在成本、延迟和质量之间给出可执行上线边界。         |

工具学习顺序：先掌握能救火的最小集合：Git、Linux、SQL、Spark UI、K8s logs、dashboard、模型评测脚本。再补平台化工具和 GPU 深水区。

# 05 | 公司标杆

从公开材料抽取共性标准

| 来源        | 公开材料里的核心信号                                                                                                  | 转化为个人技能                                                   |
|-----------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Google    | Rules of ML 强调先做稳健 pipeline、简单模型、指标、训练服务一致性和监控；SRE 强调 SLO、错误预算、告警和事故复盘；Engineering Practices 强调小变更、测试和可审查性。 | 先把数据和服务链路打稳，再追复杂模型。任何算法改动都要有指标定义、测试、回滚和运行证据。              |
| Meta      | 推荐与广告系统的鲁棒性围绕模型 snapshot、特征、训练数据、校准和解释性建立防线；Sapling 体现大规模仓库下的源代码协作效率；Faiss 体现向量检索工程能力。                      | 学会给特征、标签、模型、校准建立质量门禁。搜索广告推荐必须理解召回、排序、向量索引和实时监控。           |
| OpenAI    | 公开岗位强调分布式系统、训练框架、Python/Rust、数据能力、可观测性和高性能 I/O；Evals 资料强调 LLM 系统必须有结构化评测。                                   | 大模型方向不能只会调用 API，要能做 eval、数据、训练/推理性能、系统稳定性和研究代码工程化。        |
| Anthropic | Safeguards ML infra 岗位强调实时和批量分类器、安全评测、监控、数据质量、低延迟推理、自动测试、部署与回滚；RSP 强调随着能力提升增加安全和组织约束。                       | LLM 工程要把安全、评测、监控、回滚视为一等公民，尤其是内容安全、自动化评估和高可靠部署。            |
| NVIDIA    | CUDA、Nsight、Triton Inference Server、TensorRT-LLM、NeMo 等文档把 GPU 计算、性能分析、推理服务、动态批处理、指标和多框架部署系统化。              | 会用 GPU 不等于懂 GPU。必须能解释吞吐、显存、kernel、通信、batch、cache 和推理服务调度。 |
| ByteDance | Monolith 公开材料展示大规模推荐的实时训练、无冲突 embedding、训练服务一体化；CloudWeGo 展示高性能 RPC、云原生微服务和企业级服务治理。                         | 推荐、广告、搜索要理解实时性和服务化。算法逻辑最终要放进低延迟、高并发、可治理的在线系统。             |

**共同点 1: 先工程后复杂度**

越是顶尖团队，越强调基础设施、数据质量、评测和发布纪律。复杂模型没有生产链路支撑时，只会放大故障面。

**共同点 2: 质量门禁前移**

不要等线上用户发现问题。样本、特征、模型 snapshot、服务配置、实验指标都应在上线前被自动检查。

**共同点 3: 评测和观测闭环**

离线评测回答“是否值得试”，线上实验回答“是否真的有效”，监控和复盘回答“是否持续可靠”。



# 06 | 岗位技能包

不同算法方向的重点不同

## LLM 训练 / 研究工程师

PyTorch/JAX distributed training data curation evals GPU profiling

重点能力：数据混合与清洗、tokenization、预训练/后训练、checkpoint、并行策略、显存优化、训练稳定性、评测集治理、实验追踪。

高频风险：数据污染、eval 泄漏、训练不稳定、吞吐低、checkpoint 不可恢复、结果不可复现。

## 搜索算法工程师

Lucene ANN query understanding ranking latency budget

重点能力：分词、倒排、语义召回、向量索引、粗排精排、相关性评测、query 改写、拼写纠错、冷启动与长尾。

高频风险：离线相关性和线上满意度不一致，召回扩张拖垮延迟，热门 query 掩盖长尾失败。

## 推荐算法工程师

recall ranking feature store real-time training diversity

重点能力：多路召回、embedding、序列建模、实时特征、粗排精排重排、多目标、探索利用、生态和安全指标。

高频风险：信息茧房、热门偏置、负反馈滞后、特征新鲜度下降、模型小波动放大为流量结构变化。

## LLM 应用 / RAG / Agent 工程师

prompt eval retrieval tool use guardrails observability

重点能力：文档解析、向量检索、rerank、上下文组装、函数调用、缓存、速率限制、LLM judge、人工评审、提示注入防护。

高频风险：召回错漏、上下文污染、权限越界、幻觉、成本失控、agent 动作不可审计。

## 广告算法工程师

CTR/CVR auction calibration budget pacing experiment

重点能力：样本延迟、归因、预估校准、出价与排序、预算消耗、反作弊、广告主与用户双边指标。

高频风险：标签延迟导致训练偏差，校准异常直接影响收入和广告主体验，短期 CTR 伤害长期生态。

## 算法平台 / MLOps 工程师

Kubeflow MLflow KServe Ray governance

重点能力：训练平台、模型注册、特征平台、服务平台、实验平台、指标平台、权限、成本、自动化质量门禁。

高频风险：平台抽象过重，研究迭代被工具拖慢；或平台过薄，关键链路都靠人工约定。



# 07 | 标准流程

把能力变成可执行动作

## 流程 A: 一次算法 PR

| 步骤    | 动作                                      | 产物                       |
|-------|-----------------------------------------|--------------------------|
| 建分支   | 从最新主干切短命分支，命名包含任务和意图。                   | feat/rank-calibration-v2 |
| 小变更   | 代码、配置、数据 schema、评测脚本分开提交，便于 review 和回滚。 | 多个独立 commit              |
| 本地验证  | 跑单测、lint、最小样本训练、固定输入服务输出。               | 测试输出和评测摘要                |
| PR 描述 | 写清问题、方案、指标、风险、灰度和回滚。                    | PR 模板                    |
| 审查响应  | 逐条处理 review，争议沉淀为决策记录。                  | 更新后的代码和注释                |

## 流程 B: Hive / Spark 离线特征

| 阶段    | 必须检查                           | 失败处理                    |
|-------|--------------------------------|-------------------------|
| 表设计   | 主键、分区、时间语义、去重、权限、生命周期。         | 没有 owner 或时间口径不清，不进入训练。 |
| 作业实现  | join 规模、倾斜、空值、默认值、数据延迟、UDF 性能。 | 先在小分区 explain 和抽样，再跑全量。 |
| 质量门禁  | 行数、覆盖率、分布、极值、新鲜度、上游分区完整性。      | 门禁失败时冻结下游训练，触发补数或回滚。    |
| 血缘与文档 | 来源表、加工逻辑、责任人、适用模型、变更记录。        | 禁止无文档特征进入共享特征集。         |

## 流程 C: 训练、评测、上线

| 门   | 问题            | 通过标准                                        |
|-----|---------------|---------------------------------------------|
| 训练门 | 本次 run 是否能复现？ | 代码 commit、数据分区、配置、随机种子、镜像、依赖和资源记录完整。        |
| 评测门 | 离线指标是否可信？     | 主指标提升，关键 slice 不退化，错误案例可解释，安全和延迟通过。         |
| 服务门 | 线上资源是否足够？     | p95/p99、吞吐、显存、冷启动、超时、降级和 autoscaling 有压测证据。 |
| 实验门 | A/B 是否能回答问题？  | 随机化单元、样本量、实验时长、护栏指标和停止规则明确。                 |
| 发布门 | 出问题怎么办？       | canary、回滚命令、owner、告警、runbook 和复盘模板齐备。       |

流程纪律：算法实验可以大胆，生产变更必须保守。任何“可能影响用户”的改动，都应该默认需要灰度、监控和回滚。

# 08 | 成熟度与训练路线

把学习变成可验收项目

## 能力阶梯

| 等级      | 标准                   | 证据                                |
|---------|----------------------|-----------------------------------|
| L1 工具熟悉 | 能按文档运行工具，理解基本概念。     | 能独立完成本地训练、SQL 查询、PR 提交和服务请求。      |
| L2 独立交付 | 能完成小型算法需求，知道如何验证。    | 有可复现实验、测试、评测报告、灰度计划和回滚方案。         |
| L3 生产负责 | 能负责线上模型或服务，处理异常。     | 有 dashboard、告警、runbook、事故复盘和质量门禁。 |
| L4 系统设计 | 能设计跨团队数据、训练、服务或实验链路。 | 有接口契约、权限模型、容量规划、SLO 和迁移计划。        |
| L5 标准制定 | 能降低团队整体复杂度，沉淀平台或规范。  | 多人复用、故障率下降、迭代周期缩短、成本下降。           |

### 30 天：基础工程闭环

- 完成 Git 小变更训练：branch、rebase、PR、review、bisect、revert。
- 完成 Linux 远程排查训练：进程、磁盘、端口、日志、GPU。
- 完成 SQL / Hive 训练：分区表、窗口函数、join、explain、数据质量检查。
- 把一个 notebook 拆成 Python 包，加入单测、lint、配置和 README。

### 60 天：离线到训练

- 实现一个 Spark 离线特征任务，包含 backfill、质量门禁和血缘说明。
- 用 Kafka 或 Flink 设计一个实时特征原型，解释事件时间和迟到数据。
- 完成一次可复现训练：固定数据、代码、配置、随机种子、artifact。
- 建立离线评测报告：平均指标、slice 指标、错误案例、风险结论。

### 90 天：服务和实验

- 把模型封装为 Docker 服务，提供 HTTP 或 gRPC 接口和健康检查。
- 在 K8s 或本地等价环境完成滚动发布、日志查看和回滚演练。
- 用 vLLM、Triton 或 KServe 跑一次推理压测，报告 p50/p95/p99、吞吐、显存和成本。
- 设计一次 A/B 实验方案，包含随机化单元、样本量、护栏指标和停止规则。
- 写一次模拟事故复盘：时间线、影响面、根因、修复、预防动作。

### 6 个月：进阶方向

- LLM：构建 eval harness、RAG 评测集、安全样本和自动化回归评测。
- 搜广推：完成召回、排序、校准、重排或实时特征的线上闭环项目。
- 平台：抽象模型注册、特征质量、服务灰度或实验报告工具。
- 系统：完成 GPU profiling、分布式训练或低延迟推理优化项目。

## 09 | 检查表

### 合并前

- PR 是否只解决一个清晰问题？
- 是否有对应 issue、实验文档或设计记录？
- 是否包含测试、评测脚本或可复现实验命令？
- 配置、schema、数据口径是否写入文档？
- 是否触碰权限、PII、密钥、日志采集或外部依赖？
- 失败时是否能快速 revert 或关闭 feature flag？

### 训练前

- 训练样本分区是否完整？
- 标签时间窗是否晚于预测时刻？
- 负样本策略是否改变业务分布？
- 特征覆盖率、分布和极值是否正常？
- 代码、配置、镜像和依赖是否版本化？
- 训练失败后是否能从 checkpoint 恢复？

### 上线前

- 离线主指标提升是否超过噪声？
- 关键 slice 是否无明显退化？
- 服务 p95/p99 是否满足 SLO？
- GPU 显存、吞吐和成本是否有余量？
- canary、灰度、回滚、降级是否演练过？
- dashboard、alert、owner、runbook 是否齐备？

### 事故中

- 先判断影响面、用户面、收入面和安全面。
- 先止血，再定位根因。
- 保留关键证据：版本、配置、日志、指标、样本、时间线。
- 区分数据问题、模型问题、服务问题、依赖问题和实验平台问题。
- 复盘输出必须包含预防动作和 owner，不只描述经过。

**上线红线：**没有评测证据、没有服务压测、没有回滚方案、没有 owner、没有监控告警，任何一个缺失都不应直接全量上线。

# 10 | 反模式

真实团队最容易积累的工程债

| 反模式              | 表现                                      | 修正方式                              |
|------------------|-----------------------------------------|-----------------------------------|
| Notebook-only 工程 | 所有逻辑在 notebook，无法测试、复用、部署和 review。      | 探索可以 notebook，生产逻辑必须模块化、配置化、测试化。  |
| 只看平均离线指标         | AUC、NDCG 或 pass rate 提升，但长尾、冷启动、安全样本退化。 | 建立 slice、错误案例、护栏指标和人审样本。          |
| 无版本数据            | 同一训练命令过几天跑不出同样结果。                       | 记录数据分区、snapshot、schema、上游版本和过滤规则。 |
| 特征穿越             | 训练使用预测时不可获得的信息，离线指标虚高。                  | 做 point-in-time join，严格检查时间语义。    |
| 训练服务不一致          | 离线特征和在线特征默认值、归一化或截断不同。                  | 共享 feature spec，建立一致性测试和线上监控。     |
| 静默退化             | 任务成功但新鲜度、覆盖率、校准或分布慢慢坏掉。                 | 把数据质量、模型质量和服务质量都纳入告警。             |
| 没有降级路径           | 模型服务超时后直接影响主链路。                         | 预留 fallback、缓存、默认策略、限流和熔断。        |
| GPU 成本黑箱         | 只申请更多卡，不解释利用率、吞吐和瓶颈。                    | 固定性能报告模板，追踪 GPU 利用率、显存、吞吐和单位请求成本。 |
| LLM eval 过薄      | 只用几个样例判断 prompt 或模型升级。                  | 建立任务数据集、自动评测、人审、回归集和安全集。          |
| 实验无护栏            | 只看点击或收入，不看延迟、投诉、留存、安全、生态。               | 每个实验必须有主指标、护栏指标、停止规则和决策记录。        |

### 个人训练题

拿一个旧项目，补齐 README、复现命令、数据口径、评测报告、上线风险和回滚计划。这个练习比刷更多模型论文更接近生产。

### 团队训练题

选一个线上模型，画出从日志到训练到服务到实验的链路图，标出 owner、SLO、告警、回滚和质量门禁。

### 面试训练题

不要只讲模型结构。讲清你如何发现问题、定义指标、构建样本、控制变量、上线灰度、监控和复盘。

资料来源详见 sources.md

# 11 | 命令速查

## Git: 审查、同步、定位、回滚

```
git status --short
git fetch origin
git log --oneline --decorate --graph -20
git diff origin/main...HEAD
git rebase origin/main
git range-diff origin/main...HEAD
git bisect start
git bisect bad
git bisect good <known-good-sha>
git revert <sha>
```

生产判断: 如果无法解释每个 commit 的意图, 无法用一句话说明回滚方式, 这个 PR 还不够成熟。

## Hive / SQL: 表、分区、执行计划

```
show partitions db.table;
describe formatted db.table;
explain select ...;
select dt, count(*) from db.table group by dt;
select count(*), count(distinct user_id) from sample;
select percentile(score, array(0.5,0.9,0.99)) from pred;
```

生产判断: 先查分区、行数、去重、空值、分布和极值, 再相信训练指标。大表先抽样和 explain, 不要直接全量试错。

## Spark: 提交、观察、定位瓶颈

```
spark-submit --master yarn --deploy-mode cluster job.py
spark.sql.shuffle.partitions=...
df.explain("formatted")
df.repartition("key")
df.cache()
spark UI: Jobs, Stages, SQL, Executors, Storage
```

生产判断: 慢不等于加机器。先看 shuffle、倾斜、broadcast、spill、executor lost、GC 和数据源扫描规模。

## Kubernetes: 服务状态与回滚

```
kubecttl get pods -n <ns>
kubecttl describe pod <pod> -n <ns>
kubecttl logs <pod> -n <ns> --tail=200
kubecttl exec -it <pod> -n <ns> -- /bin/bash
kubecttl top pod -n <ns>
kubecttl rollout status deploy/<name> -n <ns>
kubecttl rollout undo deploy/<name> -n <ns>
```

生产判断: 先看事件、探针、资源限制、重启次数、镜像版本和配置变更, 再判断是不是模型本身的问题。

## GPU / 推理: 利用率、显存、延迟

```
nvidia-smi
nvidia-smi dmon
nvidia-smi pmon
nsys profile -o trace python serve.py
curl -s http://service/metrics | head
watch -n 1 nvidia-smi
```

生产判断: GPU 利用率低可能是数据加载、CPU 前后处理、batch 太小、同步等待、网络或调度瓶颈, 不一定是模型太小。

## 日志与指标: 快速缩小范围

```
rg "ERROR|WARN|timeout|oom|nan|inf" logs/
jq '.latency_ms, .model_version, .error' app.log
curl -I http://service/healthz
curl -s http://service/metrics | rg "latency|gpu|queue"
du -sh *
df -h
```

生产判断: 先按时间线关联版本、配置、流量、数据分区、上游依赖和告警, 不要只看单条报错。

# 12 | 交付模板

让生产工作可审查、可复用

## 算法 PR 最小模板

背景：

- 用户问题 / 业务问题：
- 本 PR 改动范围：

方案：

- 模型 / 特征 / 数据 / 服务变更：
- 不做的事情：

验证：

- 单测：
- 离线指标：
- slice 结果：
- 压测或资源变化：

风险：

- 数据风险：
- 线上风险：
- 安全 / 隐私风险：

发布：

- 灰度计划：
- 监控链接：
- 回滚方式：
- owner：

## 离线特征说明模板

特征名：

业务含义：

entity / key：

时间语义：

来源表：

加工逻辑：

默认值与缺失处理：

更新频率：

离线覆盖率：

在线来源：

owner：

变更记录：

## 模型上线说明模板

模型版本：

训练代码 commit：

训练数据 snapshot：

配置文件：

离线评测摘要：

安全 / 合规检查：

服务镜像：

资源申请：

SLO：

压测结论：

A/B 实验配置：

上线窗口：

回滚命令：

值守人：

## 事故复盘模板

影响范围：

用户影响：

业务影响：

开始时间：

发现时间：

止血时间：

恢复时间：

时间线：

- HH:MM 现象
- HH:MM 操作

根因：

直接原因：

深层原因：

修复：

已完成：

待完成：

预防动作：

owner 与截止日期：